The Phase-7 Engineering Constitution

Governing Principles for Development of the Structural Quant Engine

---

What the engine is: **an analytical and decision-support tool. It reads public market data and helps a trader judge an entry price, three targets, a stop, and whether the trade's risk is acceptable. It holds no credentials, sends no orders, and has no capacity to execute a trade or move money — see Items 1 and 18. It is not a trading system, and a reader assessing this document against the standards owed by one will reach conclusions calibrated to a product that does not exist.**

Status: **RATIFIED — v1.0, Revision 8 content, ratified August 26, 2026**

Date: **August 26, 2026**

Scope: **21 Tier 1 invariants, 7 Tier 2 principles, 10 Tier 3 process items, 6 Tier 4 preferences — a 44-rule register, now frozen. Revision 6 was the first revision to change this count, permitted only because the scope freeze begins at ratification and ratification had not yet happened. Revisions 7 and 8 changed no rule. The document is now ratified: this register does not change again until the audit runs and finds a gap.**

Origin: **Synthesized from a joint principles discussion between Viktor and Claude (Cowork); revised after independent review by three AI assessments (Microsoft Copilot, Google Gemini, OpenAI ChatGPT) and their cross-referenced synthesis, revised again after a fourth, independently-requested review by Claude (Opus 5) in a separate session, revised again after a philosophical input on epistemics and market structure, revised again to correct scope drift Viktor caught in that revision, revised again to add credential-security invariants and name an independent auditor before ratification, revised again to credit Reviewer 4's identity and name what it implies, and revised a final time to name Grok as Viktor's decided choice for that auditor**

What changed in Revision 1: see **“Improvements Made in This Revision & Reviewer Impressions,”** immediately after the contents page. Revision 2 added one new explanatory section — **“The Scope Freeze.”** Revision 3 fixed a self-referential evidence claim, tightened one invariant's wording, and added audit-independence safeguards. Revision 4 sharpened two rationale texts and added five new Future Amendment Candidates. Revision 5 corrected a rationale text that had drifted toward implying the engine should size positions. Revision 6 added Tier 1 Items 18–21 and gave the audit a named independent auditor. Revision 7 named Reviewer 4 as Claude (Opus 5) and used that fact to sharpen the independent-auditor discussion. Revision 8 named Grok as the intended auditor, per Viktor's decision — see Version History for the full account, **Phase7_Credential_Security_Protocol.pdf** for the operational security detail, and **Phase7_Tier0_Companion.pdf** for the philosophical material behind Revision 4.

How to read this document: **small numbers like this[1] point to plain-language notes in the Glossary at the very end** — added for a reader without a formal engineering background.

This document exists because backtesting broke this engine before. This is the third build. Roughly forty test runs and validations went into getting here. Before backtesting is attempted again, the engine needs rules it cannot quietly bend under its own weight — this is that document, now revised eight times after outside review, and, as of August 26, 2026, ratified.

Contents

Improvements Made in This Revision & Reviewer Impressions

Viktor asked three independent AI assessments to evaluate the original v1.0 draft, then had those three cross-referenced into a synthesis document — that became Revision 1. Revision 2 added one explanatory section at Viktor's own request. Viktor then commissioned a fourth, independent review of the revised document — sharper than the first three — and Revision 3 responded to it directly. Viktor then brought a philosophical input on epistemics and market structure, asked what could be learned from it, and Revision 4 is the result. Viktor then read Revision 4 and caught a rationale text drifting toward scope it shouldn't have implied — Revision 5 corrects it. Revision 6 changed the rules themselves: four credential-security invariants Viktor called for directly, and a named independent auditor closing the gap the fourth review opened. Revision 7 changed no rule — Viktor confirmed Reviewer 4's identity, and that fact turned out to bear directly on the independent-auditor discussion Revision 6 had just written. Revision 8 also changed no rule — Viktor decided who that auditor will actually be. These pages record exactly what changed across all eight revisions, what deliberately didn't change, and a candid, source-attributed impression of what each input actually contributed.

What changed in Revision 1

1 Tier 0 Wording Fixed — CHATGPT

The original text read “Four tiers, in strict order of authority,” immediately followed by a header introducing “TIER 0” — technically a fifth thing inside a stated four-tier count. Reworded to “A governing purpose, followed by four tiers, in strict order of authority.” This was the one genuine defect any of the three reviewers found in the document's own text — credit to ChatGPT for catching it.

2 Audit Given an Explicit Starting Gate — GEMINI

The Next Steps section now names a “Minimum Viable Audit”[38] — Items 2, 3, and 6 checked first, before the full 17-item sweep — since a failure in any of those three would undermine trust in every other finding. This doesn't change the audit sequence itself, only the order within Step 3. Credit to Gemini.

3 A Defined Structure for Audit Findings — CONSENSUS

Next Steps now spells out exactly what fields an audit finding should record: Principle, Status, Severity[35], Evidence, Impact, Required action, Verification, Re-audit. This operationalizes the “record Compliant / Non-compliant / Unknown” step that was already agreed on, rather than leaving it as free-text notes. ChatGPT and Copilot proposed close variants of this independently, without seeing each other's work — that convergence is why it made the cut.

4 Two Scoping Notes Added for the Audit — CHATGPT

First, a working rule that “confidence is not probability”[36] unless empirically calibrated as one. Second, a note that Item 9's distinction between a measurement, a derived quantity, a heuristic score, a model output, and an interpretation gets resolved during the audit itself, under Item 8 — not by adding a new invariant now. Both from ChatGPT, and both are audit-time guidance rather than document changes.

5	A Lightweight Amendment Note Added	GEMINI

Version History now includes a short paragraph describing, briefly, how a future v1.1 would actually get proposed and reviewed — so that process doesn't have to be invented from scratch whenever the scope freeze eventually lifts. Credit to Gemini for flagging that this was missing.

6	Six Future Candidates Logged, None Adopted	COPILOT

Copilot independently drafted six complete candidate principles for a future v1.1. None were added — the frozen-scope rule Viktor set is respected exactly as written — but they're now recorded in a new appendix page so they aren't lost before the audit is far enough along to revisit them.

7	Explanatory Glossary Added	VIKTOR

Every technical term in this document now has a small numbered marker, like this one[10], pointing to a plain-language explanation in the new Glossary at the very end — Viktor's own request for this pass, not something any of the three reviewers raised, since none of them were asked to consider a non-engineer reader.

What changed in Revision 3

A fourth review, requested independently of the first three, read this document more critically than any prior pass — including finding a defect in how this very page talked about the first three reviewers. Every item below responds directly to that review.

1	Self-Referential Evidence Removed from This Page	REVIEWER 4

This page previously argued that three reviewers finding nothing structurally wrong was “a real signal the original drafting session got the substance right” — treating correlated agreement as validation, which is precisely what Tier 1, Item 11[12] and Item 7 forbid the engine itself from doing. Both instances of that reasoning, below and in “Impressions,” are rewritten to state plainly that reviewer agreement is a plausibility check, not proof — credit to Reviewer 4 for catching the document violating its own rules.

2	Item 5 Reworded from Preference to Invariant	REVIEWER 4

Reproducibility[7] read “every analysis *should* be reconstructable” and “...*should* all be recoverable,” the only Tier 1 item written as a preference in a list where every other item says must or never — exactly the kind of drift Items 9 and 10 exist to catch. Both instances changed to “must.” Wording only; the requirement itself was already understood to be non-negotiable.

3 Escape Hatches on Items 4 and 12 Given Audit-Time Guidance — REVIEWER 4

Determinism[6] and Hidden State[13] both carry a legitimate exception clause — “unless nondeterminism is explicitly and deliberately intended” and “...an explicit, documented part of the design” — but neither said who authorizes an exception or where it has to be recorded, leaving both unfalsifiable at audit time. Fixed with audit-time guidance, not a new rule: an exception only counts if it was documented before the audit began. Anything claimed afterward is Non-compliant. See the rationale text under Items 4 and 12.

4 The Freeze-Lifting Trigger Now Has a Definition — REVIEWER 4

“The audit has run” unfreezes this document's scope, but the phrase itself was never defined — an ambiguous trigger on the rule that governs every other rule's stability. Now defined in Next Steps: the freeze lifts once every Tier 1 item has a recorded finding — Compliant, Non-compliant, or Unknown — regardless of whether fixes have landed yet.

5 Ratification Given a Mechanical Definition — REVIEWER 4

Ratification[3] gates the entire audit sequence but had no stated act that actually performs it. Now defined in Next Steps: Viktor writes “ratified,” and a dated row is added to Version History. Nothing heavier than that.

6 Severity Levels and an Effort Field Defined — REVIEWER 4

The finding schema named Critical / Major / Moderate / Minor[35] with no rubric, and Step 5 of the audit sequence sorts findings by “severity and effort” even though effort was never a field in the schema. Both fixed: each severity level now gets a one-line definition, and Effort joins the finding schema table in Next Steps.

7 Audit Independence Named and Mitigated — REVIEWER 4

The most important finding: Claude holds the technical and architectural judgment calls, wrote much of the engine, co-drafted these rules, and would also be the party auditing the engine against them — grading its own homework, never named anywhere in this document. Now named directly in Next Steps, with three concrete safeguards: every finding's Evidence must be independently verifiable without re-running Claude's reasoning; the Minimum Viable Audit[38] gate items get a second check from a reviewer who didn't write the code; and where Claude marks something Compliant, Viktor can demand the artifact, not the argument.

What changed in Revision 4

Viktor asked a separate question — not “is this document correct,” but “what philosophical questions should I be asking myself about the engine” — and brought back an input worth taking seriously. Applying the input's own stated test (a question earns its place only if answering it differently produces a different engine), five of its points were concrete enough to act on now; two more sharpened rationale text already in this document; the rest sit above the engine and now live in a new companion document instead.

1	Controlled Changes Rationale Sharpened	PHILOSOPHY INPUT

Tier 3's Controlled Changes principle already does the right thing; it never explained why. Added: the Duhem-Quine problem[40] — a failed test alone never says which component failed — is the actual reason a change has to stay narrow enough to isolate one candidate cause.

2	Item 14 Rationale Sharpened	PHILOSOPHY INPUT

“Risk Is Not Conviction” was already correct; ergodicity[41] is the mathematical reason why — a positive-expected-value strategy can still ruin you with near certainty, because you live one path, not the average of all possible ones. Cross-referenced directly to the configurable-risk-tolerance feature idea in Engineering Notes, Entry #5: it's the reason entry/stop/target price geometry deserves as much engineering care as the directional read. It is not a reason for the engine to size positions or recommend money amounts — that stays the trader's decision alone, per Item 1.

3	Five New Future Amendment Candidates	PHILOSOPHY INPUT

A written kill condition[42] decided before a backtest runs; a confidence calibration log[43] that turns “confidence is not probability” from a definition into a checked fact; a requirement that every claimed edge state a hypothesis for why it should persist (reflexivity[44]); UI-level enforcement of Item 1 so the panel can't relay a verdict without showing what was observed, inferred, and unknown (the moral crumple zone[45] risk); and a two-analyst test for whether a Tier 1, Item 9 quantity is a measurement or a lens. All five recorded in Future Amendment Candidates below — none adopted, per the frozen scope.

What changed in Revision 5

1	Item 14 Rationale Corrected — Position-Sizing Drift Removed	VIKTOR

Revision 4's ergodicity rationale under Item 14 drifted into implying the engine should have a hand in position sizing or money-amount recommendations. It shouldn't have implied that, and Viktor caught it directly: the engine's output is the entry price and the T1/T2/T3 target prices, plus a read on the safety and quality of the trade — never how much money is put on, or should be put into, a specific trade. The engine does take the trader's chosen risk-tolerance percentage as an input, per the configurable-risk-tolerance feature idea in Engineering Notes, Entry #5, and uses it to shape where entry, stop, and target prices sit — but how many dollars that percentage represents, and whether to trade at all, stays the trader's decision alone, per Item 1. Item 14's rationale text below is corrected accordingly. The Revision 4 box above is left exactly as it was written — this box documents the correction rather than rewriting history.

What changed in Revision 6 — the first real scope change

Every revision before this one ended with the same sentence: scope unchanged, still 17 / 7 / 10 / 6. This one doesn't. Revision 6 adds four Tier 1 invariants and names a party that was previously unnamed, which makes it the first revision since v1.0 to change what the rules actually are rather than how they're worded. That is legitimate for exactly one reason, and it's worth being precise about it: the scope freeze[4] begins at ratification, and

this document is not ratified. After Viktor writes “ratified,” a change of this size would require the audit to run first and turn up a gap. Before that moment, it costs nothing. That timing is the whole reason both items below are being handled now rather than filed as Future Amendment Candidates.

1	Four Credential-Security Invariants Added to Tier 1	VIKTOR

Items 18 through 21: read-only market access stated categorically rather than as a default; withdrawal permissions never enabled, kept as a separate floor that survives any future amendment to Item 18; credentials never exposed through the specific channels they actually leak from; and operator credentials never conveyed to the project, written before any multi-user capability exists rather than after. The strongest of the four is Item 18, and not because of what it forbids — a read-only key[19] moves Item 1's guarantee out of the code and into the exchange, where the engine cannot overrule it. That converts “tool, not autonomous actor” from a behavioral promise into a structural property. This closes the gap logged as Engineering Notes Entry #1 and the Item 1 credential check flagged by the fourth reviewer in Entry #6. Operational detail — storage, rotation, revocation, allowlisting, incident response — lives in the companion Phase7_Credential_Security_Protocol.pdf, deliberately, so this document stays a register of things that must never be violated rather than an operations manual.

2	The Audit Given an Independent Auditor	VIKTOR

Next Steps specified what gets checked, in what order, and how findings get recorded — but never who does the checking, which left the fourth reviewer's audit-independence finding mitigated rather than solved. Steps 3, 4, and 8 now belong to an independent auditor[46] that wrote neither this document nor the engine. Claude prepares the package and writes zero findings; the auditor produces them; Claude answers each one adversarially, including against its own work; Viktor adjudicates. Two things are stated explicitly rather than assumed: the auditor is independent in authorship but not presumed independent in judgment[47], and the package carries failure *classes* rather than conclusions, so the audit tests hypotheses instead of confirming hints.

What changed in Revision 7 — a name, and what it means

Revision 6 left Reviewer 4 unattributed and asked Viktor to credit it by name if he knew the source. He did: Claude, Opus 5, in a separate chat session with no memory of co-drafting this document.

1	Reviewer 4 Identified — and the Independent-Auditor Section Sharpened	VIKTOR

The name alone would be a footnote. What it implies isn't. Reviewer 4 — already credited, before this revision, as the sharpest and most valuable of the four — turned out to share its base model family with the assistant whose document it was auditing, and with the assistant writing this sentence. It had genuine authorship independence: it didn't write these rules or this engine, and it caught a defect none of the other three thought to check. Revision 6's distinction between authorship independence and judgment independence[47] was written as a general caution about how the independent auditor for Steps 3, 4, and 8 might turn out. It is no longer only that — Reviewer 4 is the documented case where exactly this happened once already, and it worked reasonably well. The “who performs the audit” section now says plainly that if the actual auditor also turns out to be Claude, that result should be weighted as weaker than a genuinely different model family would produce, not treated as equivalent. Filed here rather than edited into Revision 6's own account of itself, per this document's practice of recording what changed as a new dated entry rather than rewriting history.

What changed in Revision 8 — the auditor named

Revision 7 sharpened a warning: if the independent auditor for Steps 3, 4, and 8 turns out to be Claude, that result carries less weight than a genuinely different model family would. Viktor answered it directly, and firmly enough that this document treats it as settled rather than pending.

1	Grok Named as the Intended Independent Auditor	VIKTOR

Viktor's own words: “I am considering implementing Grok 4.7 to be the sole auditor when it releases. In fact i know i will. It is decided.” Recorded in Next Steps as the current plan rather than folded into the independent-auditor requirement itself, for a reason worth stating plainly: a vendor and version number is an operational choice, not an invariant, and this document's own discipline throughout — the Credential Security Protocol split out for exactly this reason, Item 18 stated categorically rather than tied to any particular exchange — has been to keep specific tooling choices revisable without requiring a constitutional amendment. What the Constitution actually requires is unchanged: a model with no stake in this engine and no shared authorship with the party that built it. Grok satisfies the harder half of Revision 7's warning — different company, different training data, genuinely different lineage from Claude. It was never evidence, on its own, of the other half: no stake in the outcome. That still has to hold regardless of which model ends up in the role. One dependency worth naming honestly: Grok 4.7 has not released as of this writing, so Step 3 cannot begin until it does and Viktor judges it capable of the task. That blocks the audit itself. It does not block ratification, and it does not block Step 2a — Claude can assemble the audit package now, on its own timeline, so the package is ready the moment the auditor is.

What still did *not* change, and why that matters

The original 17 Tier 1 invariants[2], the 7 Tier 2 principles, the 10 Tier 3 process items, and the 6 Tier 4 preferences are word-for-word the same rules as before. Nothing was rewritten, reordered, softened, or removed across eight revisions — the four new invariants were appended as Items 18 through 21 rather than woven in, for the same reason Item 17 was appended rather than reordered: renumbering would silently invalidate every cross-reference in the document. Between four reviewers, the substantive critique has concerned phrasing, sequencing, future scope, and — from the fourth review — this document's own epistemic conduct, not the content of the original 40

rules. That's worth stating honestly rather than as more than it is: four separately-prompted reviewers finding no rule they consider structurally wrong is a plausibility check, not a validation. They are not independent of each other — overlapping training data, the same prompt, the same document — and they are not independent of the document's co-author. Correlated agreement is weak evidence of correctness. It is worth having. It is not proof, and this document no longer claims otherwise.

Impressions of the reviewers

Gemini was the most concise and, word for word, the most operationally useful — the Minimum Viable Audit ordering is a genuinely good idea, delivered in two pages instead of twenty. It also deserves credit for immediately recognizing Item 17 as evidence of real operational scars rather than textbook caution, which is exactly the right read.

ChatGPT did the deepest structural reading of the three, sorting all 17 Tier 1 invariants into failure-class categories as an unprompted exercise, and it found the one genuine defect in the document's own wording. Its proposed audit-finding structure is probably the single most practically useful addition in this revision. It also showed real judgment restraint — recommending the Item 9 nuance be handled during the audit rather than inflating the document with a new invariant just because it noticed something.

Copilot was the most generous and ambitious reviewer, going well beyond what was asked by drafting six complete future amendment candidates on its own initiative — real engineering effort, not just commentary. Two of the six turned out to be well-aimed, converging with content already in this document. Its rocky start is worth naming honestly rather than glossing over: its first response claimed the file couldn't be read, and only produced a real assessment after Viktor pasted the text in directly — which is part of why some of its critique (particularly around Claude's authority) was flagged with a caveat in the synthesis document rather than taken at face value. Worth noting in hindsight: that's the one external critique aimed at Claude's authority that got discounted — and the fourth review below suggests that may be a pattern worth watching, not just a one-off judgment call.

Reviewer 4 (Claude, Opus 5, in a separate chat session with no memory of drafting this document) did the least generous and most valuable review so far. It didn't sort invariants into categories or draft new material; it audited this document's own conduct and found it wanting by the exact standards the document sets for the engine. Catching this page's self-referential evidence claim, and naming the audit-independence gap outright, is exactly the kind of finding that's easier to produce with real distance between reviewer and author — which is itself a small piece of evidence for the point it's making. The wording defects and undefined triggers it also caught (Item 5's stray “should,” the unowned escape hatches on Items 4 and 12, an undefined freeze-lifting condition, an undefined act of ratification, an unrubric'd severity scale) are individually small, but finding five of them in one pass, after two prior revisions each reviewed by someone, says something about how finished a document can look while still not being finished.

Overall: four reviewers, four different kinds of value. The first three found one real wording defect and several genuinely useful process refinements, and were generous with the document. The fourth found sharper problems: an unearned claim on this very page about what reviewer agreement proves, and a structural gap none of the first three named — that the party co-drafting these rules and largely writing the engine is also the party positioned to audit it. Both are corrected here rather than defended. That's the actual point of running an audit process and a scope freeze in the first place: the document needs to be able to survive being told it's wrong, not just collect agreement.

The philosophy input (source not yet attributed in this document) wasn't reviewing the Constitution at all — Viktor asked it a different question, “what philosophical questions should I ask myself about this engine,” and brought the answer back here to see what applied. That's a different and useful kind of contribution: not auditing what's already written, but testing whether the document's philosophical foundations are as solid as its process. Applying its own stated filter honestly meant most of its ten questions turned out to be about Viktor and the project, not the engine — and those correctly stayed out of this document rather than being forced in to look thorough. The five that passed the filter are real, checkable additions to the audit's future work; the rest are in Phase7_Tier0_Companion.pdf, where they belong.

How This Document Works

A governing purpose, followed by four tiers[1], in strict order of authority. Tier 1 cannot be violated by a prettier result. Tier 2 is how the system must be shaped to make Tier 1 achievable. Tier 3 is how changes get made without breaking either. Tier 4 is legitimate taste — real trade-offs reasonable engineers could weigh differently, and the only tier allowed to bend.

TIER 0 — GOVERNING PURPOSE

> **The engine must never be optimized to appear intelligent. It must be optimized to produce reliable, testable, interpretable information and decisions under real-world conditions.**

This isn't a rule to check code against. It's the reason the other rules exist, and the test for any future suggestion — including ones I make myself. If a proposed change makes the engine sound smarter without making it more reliable, testable, or interpretable, it doesn't belong, no matter how impressive it looks.

The development sequence, once this is ratified[3]:

Principles v1.0 → version/freeze → audit the current engine → record Compliant / Non-compliant / Unknown[34] for each item checked → prioritize findings → fix deliberately → regression test[26] → re-audit → only then build the backtesting[39] architecture, carefully. The principles do not move while the engine is being judged against them — inventing the rules and grading the system by rules still in motion would itself violate epistemic honesty[10] (Tier 1, Item 8).

“Unknown” is a legitimate audit result — not a placeholder for laziness, a real outcome. If it isn't yet clear whether a module violates a principle, the honest answer is Unknown, pending inspection — never compliant or non-compliant based on a guess. This document already uses that status once, on purpose: see Layer 5 in the Next Steps section.

Roles & Authority

This document assigns Claude the technical and architectural judgment calls. That is not blind authority, and it isn't meant to read as one. The working relationship is: Claude proposes, reasons, challenges, implements, and tests; Viktor reviews, questions, performs QA, and ultimately accepts or rejects the change. Neither side gets to declare something true merely because it sounds reasonable — evidence gets the final vote, on both sides of that relationship, including when the evidence contradicts a judgment call made in this very document.

The Scope Freeze[4]

The frozen-scope rule says the list of rules in this document stops growing the moment it's ratified, and stays stopped until one specific thing happens: the audit (Next Steps) actually runs against the current engine and turns up something not yet covered here. It isn't frozen forever — it's frozen until that particular trigger, not “whenever it seems like a good idea.”

The reason ties directly back to epistemic honesty[10] (Item 8). Once real code is being checked against these rules — each item marked Compliant, Non-compliant, or Unknown[34] — the rules themselves have to hold still. Being allowed to invent a new rule, or reword an existing one, mid-audit would make it possible to quietly turn a Non-compliant finding into a Compliant one by moving the goalpost instead of fixing the code. That's grading a test while still writing the questions. The freeze is what makes an audit finding mean something.

What's actually frozen is narrow: the count and substance of the rules — 21 Tier 1 invariants[2], 7 Tier 2 principles, 10 Tier 3 process items, and 6 Tier 4 preferences. Nothing gets added, nothing gets removed, until the audit says otherwise. What isn't frozen is wording, phrasing, and explanatory material — which is how this document could be revised five times (see Version History) without breaking its own rule: no revision through Rev 5 changed what any rule requires.

One point of precision, since Revision 6 did change the count. The freeze does not start when the document is written — it starts when it is **ratified**[3], and that hasn't happened yet. That's why four credential invariants and a named independent auditor could be added in Revision 6 without violating anything: there was no freeze in force to violate. The moment Viktor writes “ratified,” that door closes, and a change of that size would require the audit to run first and turn up a gap. Anything still wanted in this document is wanted *now*, not after.

When the freeze does lift, the Future Amendment Candidates appendix — currently six drafted principles, none adopted — becomes the starting pool for a genuine v1.1, reviewed the same lightweight way described under Version History: a short note on what changes and why, Claude proposes, Viktor decides, a new dated row. Not adopting a good idea today isn't a judgment on the idea. It's the same rule working in reverse — changing the rules before checking the engine against the ones already agreed on would be exactly the violation this rule exists to prevent.

Tier 1 — Fundamental Invariants

These cannot be violated because doing so produces a better-looking result. A change that requires breaking one of these isn't a trade-off to weigh — it's disqualified before the rest of the conversation starts.

1 Tool, Not Autonomous Actor — INVARIANT

"The engine is an analytical and decision-support system. It must never independently execute trades or otherwise take irreversible external action."

Why it's non-negotiable: Analysis ≠ authority. This is the one invariant that isn't about whether the analysis is good — it governs what the system is allowed to do with it regardless of how confident it becomes. A 99.7-confidence read still doesn't get to touch an exchange.

2 No Future Information / Look-Ahead Bias — INVARIANT

"Information unavailable at the exact decision timestamp must never influence that decision, whether directly or through a derived signal."

Why it's non-negotiable: This is the invariant backtesting infrastructure most commonly violates by accident — a moving average computed with a centered window, or an indicator quietly peeking one bar ahead, can make a system look prescient in testing and useless live. Given backtesting is the next major build, this is the invariant to protect most carefully. Minimum Viable Audit[38] status: checked in the audit's first gate, before the full sweep — see Next Steps.

3 Data Integrity — INVARIANT

"The engine must actively distrust its inputs. Missing candles, duplicated candles, impossible prices, timestamp inconsistencies, NaN/Inf values, stale data, malformed API responses, and abnormal volume must be detected before they become analysis."

Why it's non-negotiable: Garbage data must never quietly become good-looking output. A confident number computed from broken input is worse than no number at all, because it doesn't announce itself as broken. Minimum Viable Audit status: checked in the audit's first gate — see Next Steps.

4 Determinism — INVARIANT

"Given identical inputs, configuration, and code version, the engine must produce the same result, unless nondeterminism is explicitly and deliberately intended."

Why it's non-negotiable: Without this, debugging becomes archaeology. If the same input can produce two different outputs on two different runs, there is no way to know whether a fix actually fixed anything. Audit guidance: the exception clause only counts if the intended nondeterminism was documented before the audit began — a module claiming this retroactively, once the audit is already underway, is Non-compliant regardless of the explanation offered.

5 Reproducibility — INVARIANT

“Every analysis must be reconstructable later — its data timestamp, data source and version, engine version, configuration, and parameters must all be recoverable.”

Why it's non-negotiable: Six months from now, “why did the engine produce this decision” needs to be an answerable question, not an archaeological one. Recording this metadata describes the input, but doesn't by itself guarantee the input can be recovered if an exchange's API later revises or rolls off historical data — whether raw pulls need to be archived, and for how long, is a retention decision the audit should force explicitly rather than leave implicit.

6 Traceability — INVARIANT

“Every major output must have an explainable lineage: decision ← decision components ← normalized signals ← raw signals ← indicators ← validated market data ← raw source data.”

Why it's non-negotiable: You should be able to walk backward through the chain from any panel output to the exact raw candles that produced it. If that walk breaks anywhere, the output can't be trusted, no matter how reasonable it looks. Minimum Viable Audit status: checked in the audit's first gate — see Next Steps.

7 No Unsupported Predictive Claims — INVARIANT

“The engine must distinguish between describing current market conditions and predicting future price behavior. Any predictive component must be empirically[16] tested before it is presented as predictive.”

Why it's non-negotiable: This is the direct rule the BTC-Adjusted AERO Prediction feature currently sits in tension with — see the Next Steps section for how that gets resolved without ripping the feature out.

8 Epistemic Honesty — INVARIANT

“The engine must distinguish, at all times, between what is directly observed, what is mathematically derived, what is interpreted, what is hypothesized, what has been empirically validated, and what remains unknown.”

Why it's non-negotiable: This is the parent principle several others are specific cases of — Item 7 is epistemic honesty applied to predictions, Item 11 is epistemic honesty applied to evidence independence. A number that blurs “hypothesized” into “validated” is dishonest even if every digit in it was computed correctly.

9 Every Measurement Has a Precise Definition — INVARIANT

“Terms such as trend strength, momentum, alignment, confidence, and risk each need an explicit mathematical or semantic[11] definition.”

Why it's non-negotiable: A number with two decimal places is not automatically meaningful. If nobody can state exactly what “confidence: 74.32” means, it doesn't matter how precise it looks. Audit scoping note: whether this term should also formally distinguish a measurement from a derived quantity, a heuristic score, a model output, and an interpretation is a decision for the audit itself, under Item 8 — see Next Steps.

10 Consistent Semantics — INVARIANT

"The same term, score, or scale must mean the same thing in every module that uses it."

Why it's non-negotiable: This is the rule the earlier confidence-score bug violated — one code path treated "confidence" as a real computed value, another silently passed through a differently-scaled number under the same name. Same word, two meanings, one bug.

11 No Circular Reasoning — INVARIANT

"A signal must not be allowed to reinforce itself through multiple derived layers and then be presented as independent confirmation."

Why it's non-negotiable: Five layers all quietly descended from the same original signal can look like five independent pieces of evidence when they're really one piece of evidence talking to itself in five accents. This is the lens the Layer 5 audit item (Next Steps) needs to be examined through.

12 No Hidden Decision-Affecting State — INVARIANT

"Modules must not secretly remember things that influence a later decision, unless that state is an explicit, documented part of the design."

Why it's non-negotiable: Hidden state is what makes a bug feel impossible to reproduce — the same inputs produce a different answer because something invisible changed between runs. Audit guidance: the same rule as Item 4 applies here — a module's state only counts as "explicit, documented design" if that documentation existed before the audit began. Otherwise the claim is unfalsifiable and the finding is Non-compliant.

13 Fail Safely — INVARIANT

"When data is missing, invalid, stale, or contradictory, the engine must become less confident or halt output — never invent a confident-looking answer."

Why it's non-negotiable: A broken data feed must never quietly produce "BUY, confidence 94." It should produce something closer to "ANALYSIS INVALID / INSUFFICIENT DATA." Confidence earned from real evidence and confidence manufactured to fill a gap must never look the same on the panel.

14 Risk Is Not Conviction — INVARIANT

"Directional conviction must never be treated as equivalent to risk. Being highly confident an asset is bullish does not automatically justify taking on high risk."

Why it's non-negotiable: Volatility, liquidity, drawdown potential, and market conditions can all make the risk/reward poor even when the directional read is strong. These are answers to two different questions, and the engine must never let one substitute for the other. Ergodicity[41] is part of why this matters as much as it does: a strategy with positive expected value can still ruin you with near certainty, because you compound along one actual path rather than averaging across the many possible ones. That's a reason to take risk/reward geometry — where entry, stop, and targets sit relative to each other — exactly as seriously as the directional signal. It is not a reason for the engine to size positions or recommend how much to risk in money terms: that decision belongs to the trader alone, entirely outside what this engine does, per Item 1. See Phase7_Engineering_Notes.pdf, Entry #5, for the configurable-risk-tolerance feature idea — it shapes entry and target *prices*, never position size.

15 Empirical Evidence Supersedes Theoretical Expectation — INVARIANT

“When live, measured behavior and mathematical elegance disagree, measured behavior wins.”

Why it's non-negotiable: A beautifully derived formula that doesn't hold up against real data doesn't get to stay in the engine out of respect for the math. This is also the invariant that makes it safe to eventually kill a feature that isn't working, without that decision being personal.

16 Complexity Must Provide Demonstrated Value — INVARIANT

“New indicators, models, calculations, or layers must exist because they solve a demonstrated problem or provide measurable value — not merely because they can be added.”

Why it's non-negotiable: More code is not a better engine. Every layer added since the original build should be able to point to the specific problem it solved; if it can't, that's a finding for the audit, not a reason to feel bad about it.

17 Backtesting Must Be Isolated From the Core Engine — INVARIANT

“A failure in backtesting[39] or other validation tooling must never be capable of silently corrupting, destabilizing, or redefining the live analytical engine. When it fails, it must be possible to isolate the failure, identify what broke, and return to the last known-good engine state without ambiguity.”

Why it's non-negotiable: The goal is not an unbreakable backtester — that's an unrealistic bar, and claiming it would itself violate epistemic honesty (Item 8). The goal is a blast radius: backtesting is allowed to fail, but it is never allowed to take the live engine down with it. This item exists because of lived history, not caution for its own sake — backtesting is what broke a previous version of this engine, and this is the invariant written specifically so that doesn't happen the same way twice. Added after the rest of this document had already been drafted, appended here rather than reordered to avoid silently invalidating every other item's cross-references — see Tier 2's “explicit, evaluated changes” entry for why that matters.

18 Read-Only Market Access — INVARIANT

“The engine must never hold, request, or have access to API credentials[20] carrying trade-execution permissions. Read-only access is not a default setting — it is the only permitted state.”

Why it's non-negotiable: Item 1 says the engine is a tool, not an autonomous actor. Until now that was enforced only by the code happening not to contain order-placement logic — one convenience import, or one future contributor, away from being false. A read-only key moves the guarantee out of the code and into the exchange, where the engine cannot overrule it: even a fully compromised engine holding a read-only key cannot place a trade, because the permission does not exist to be misused. That converts Item 1 from a behavioral promise into a structural property, which is the entire point. This is stated categorically rather than as a default on purpose. A default invites an exception, and an undocumented exception is precisely the defect the fourth review found in Items 4 and 12. If an execution capability is ever genuinely wanted, it is a different product with a different risk profile, and it should cost a constitutional amendment through Tier 3 change control — not a configuration flag.

19 Withdrawal Permissions Are Never Enabled — INVARIANT

“No credential the engine holds, touches, or is configured with may carry withdrawal, transfer, or fund-movement permissions. This holds in every version, under every circumstance, with no exception.”

Why it's non-negotiable: Item 18 already excludes withdrawal rights, so on the surface this is redundant. It is stated separately because it is the floor that survives Item 18. If some future amendment ever relaxes read-only access to permit execution, this item does not relax with it — withdrawal rights still never exist. Separating them means a future change to one cannot quietly carry the other along with it. The asymmetry in consequences justifies the redundancy: a wrong trade loses money that was already at risk in the market, while a withdrawal permission in the wrong hands empties an account entirely, and no analytical benefit anywhere in this engine's purpose requires that permission to exist.

20 Credentials Are Never Exposed — INVARIANT

“API keys, secrets, and any material that authenticates to an exchange must never be hardcoded, committed to version control, written to logs, included in error messages or stack traces, or surfaced in any panel, report, or decision-support output.”

Why it's non-negotiable: This is the invariant that most often fails by accident rather than by decision. Nobody commits a key on purpose; it arrives through a debug print left in, an exception handler that dumps its configuration, a screenshot taken during a working session, or a config file that was in the repository before anyone thought about it. Because the failure mode is inattention rather than intent, the rule has to name the specific channels rather than say “keep credentials safe,” which is unauditable. Each channel named above is a thing the audit can actually check. Note the connection to Item 6 (Traceability): the engine is required to make its reasoning inspectable, and this item marks the one category of state that is deliberately exempt from that requirement — the audit trail records that a credential was used, never what it was.

21 Operator Credentials Stay With the Operator — INVARIANT

“If this engine is ever run by anyone other than its author, that operator's credentials remain entirely within their own environment. The engine must never transmit, upload, centralize, or otherwise convey any operator's credentials to the project, its author, or any third party — including indirectly, through telemetry[24], crash reports, error reporting, usage analytics, or support workflows.”

Why it's non-negotiable: The other three credential invariants protect the operator from their own tooling. This one protects a third party from this project, and the distinction matters: a leak here harms someone who trusted the engine rather than the person who built it, which makes it a categorically different failure and an irreversible one. It is written now, before any multi-user capability exists, deliberately. A security invariant that arrives after the feature it constrains is how credential leaks happen — the capability gets built, the rule gets written to describe what was built, and the convenient shortcut is already load-bearing by then. Written first, it constrains the design instead of ratifying it: no credential-upload path, no central key store, no support flow that asks an operator to share a key, and no diagnostic channel capable of carrying one out.

Tier 2 — Architectural Principles

How the system must be shaped so that Tier 1 is actually achievable, not just aspired to. These can be redesigned if a better structure achieves the same invariants — but the invariants themselves don't move.

Principle	What it means

Separation of responsibilities	Data acquisition doesn't make decisions. Indicators don't manage risk. The decision layer doesn't silently redefine raw signals it receives.
Observation → interpretation → decision → risk/action	EMA20 > EMA50 is an observation. “Short-term trend is bullish” is an interpretation of it. These must stay distinguishable stages, never one function that quietly does all four.
Explicit interfaces/contracts[25]	What a module accepts and returns should be a stated contract, not something the next module has to reverse-engineer by reading the source.
Explicit configuration	Important behavior (RSI period, EMA windows, confidence caps) comes from named, visible configuration — not mystery constants buried in the logic that touches them.
Modular design	Each module has one clear job. This is also why bugs in this project have generally been findable — the boundaries between pieces have stayed real.
Controlled dependencies	What depends on what should be an intentional, known map — not something only discovered when a change somewhere else breaks something unrelated.
Explicit, evaluated changes to interfaces or behavior	Changes to interfaces, semantics, outputs, or behavior must be explicit, and their downstream effects evaluated before acceptance — the rule that would have caught a silent confidence_score → confidence rename before it broke fourteen modules downstream.

Tier 3 — Engineering Process

How changes actually get made without silently breaking Tier 1 or Tier 2. This tier is process, not architecture — it governs the act of changing the system, not its shape.

Principle	What it means
Hypothesis-driven development[27]	Problem → hypothesis → implementation → test → measurement → evaluation → accept/reject. Never “make it smarter.”
Controlled changes	One meaningful change, or one tightly defined group of changes, at a time — with what's changing and why stated before the work starts. This is the practical defense against the Duhem-Quine problem[40]: a failed test never says on its own which piece failed — the indicator, the data, the sizing, an unstated assumption — so the change has to be narrow enough that there's only one real candidate. Decide which component is being tested before the result comes back, not after.
Automated tests	Every meaningful modification gets tested before it's accepted, not after.
Regression tests[26]	A new improvement must not silently destroy previously working behavior. Existing tests and fixed datasets exist specifically to catch this.
Fixed evaluation datasets	A stable, known dataset changes get measured against, so “better” and “worse” mean something consistent from one change to the next.
Version control	Real history, not manually zipped backup folders — this is also Step 3 of Viktor's own career roadmap.

Known-good checkpoints	Preserve working versions before significant modifications. Every major iteration should be recoverable, not just the current one.
Reproducible experiments	Someone else — or future-you — should be able to rerun a validation test and get the same measured result.
Rollback capability[28]	If a change makes the system worse, revert it. Time already spent building something is not a reason to keep a change that made things worse.
Documentation of significant decisions	Why a change was made, not just what changed — the practical instrument that makes Traceability (Tier 1, Item 6) actually real six months later. A dated Architectural Decision Record[29] is one concrete way to do this.

Tier 4 — Engineering Preferences

Legitimate trade-offs, not invariants. Reasonable engineers could weigh these differently in a specific situation — that's what makes this tier different from the other three, and why it's the only one allowed to bend.

Preference	What it means
Robustness over optimization[31]	A design that holds up across varied conditions beats one finely tuned to perform best in one narrow condition.
Generalization over historical fit[32]	Especially relevant given the engine currently focuses on one asset — a result that only works for AEROUSDT's specific history is weaker than one that would hold on a different symbol too.
Explainability over unnecessary opacity[33]	Avoid opaque logic merely because it produces attractive-looking results.
Stability over flashy outputs	A dependable, boring number is worth more than an exciting, unreliable one.
Useful information over more information	Adding another score to the panel is not automatically an improvement.
When multiple designs already satisfy every invariant equally well, prefer the simpler one	This is a tie-breaker, not a requirement — Tier 1, Item 16 already requires complexity to earn its place; this only decides between options that have already cleared that bar.

A Note on Backtesting[39]

This section exists because of context that changes how carefully the next major build needs to be handled: this is the third build of this engine, roughly forty test runs and validations went into reaching the current version, and backtesting specifically is what broke a previous one.

Backtesting was never off the table — it remains a real, planned priority, and Tier 1, Item 7 and Item 15 both depend on it eventually existing. What changes is the sequencing: given its track record, backtesting should not be attempted again until the Tier 3 safety net around it is real rather than aspirational — actual version control, actual known-good checkpoints, an actual fixed evaluation dataset. Building the safety net first is not delay for its own sake; it's the difference between a second failure being a recoverable setback and a fourth rebuild.

The goal is not an unbreakable backtester. That bar is unrealistic, and this document isn't going to pretend otherwise. The goal, precisely, is Tier 1 Item 17: a failure in backtesting must never be capable of silently corrupting, destabilizing, or redefining the live analytical engine. When it fails — and it may — the failure must be isolable, its cause identifiable, and the engine recoverable to its last known-good state without ambiguity. That is a containment requirement, not a perfection requirement.

When backtesting work does begin, it should be treated as a major architectural implementation in its own right, not another feature slotted into the existing build rhythm. Tier 3's hypothesis-driven and controlled-change principles apply to it more strictly than to anything else built so far — a precisely scoped definition of what's being tested and how correctness will be checked, built and verified incrementally, with each increment tested and checkpointed before the next one starts, rather than one large implementation attempted in a single pass. Tier 1, Item 2 (no look-ahead bias) deserves the most explicit, deliberate checking of any invariant in this document once that work starts, since a backtest with a hidden look-ahead leak is the single most common way a system convinces itself it works when it doesn't.

Next Steps: The Audit Sequence

This document isn't ratified[3] yet. Once it is, here's what happens, in order — two items are already known well enough to flag now, and this revision adds three concrete process refinements the original draft left implicit.

Already known — Prediction V1's epistemic status. The BTC-Adjusted AERO Prediction feature is correctness-validated (it computes what it was designed to compute) but not empirically validated (nothing has shown it actually predicts anything yet). Under Tier 1, Item 7, that status needs to be stated honestly rather than implied away — something like “Prediction V1: computationally validated, empirically unvalidated”[37] wherever the feature is presented. This doesn't require removing the feature. It requires being honest about what it currently is.

Already known — Layer 5, classification: Unknown. Entry quality's multipliers combine macro_bias, trend_direction, and structure_sequence into one score. Whether any of those three were already partly derived from each other upstream — which would make this a Tier 1, Item 11 (no circular reasoning) violation — has not actually been checked against the real code yet. Per this document's own rule that Unknown is a legitimate result, that's exactly what this is classified as until the audit inspects it directly.

New — the audit's starting gate (Minimum Viable Audit[38]). Rather than treating all 21 Tier 1 invariants as an equally-weighted checklist, Step 3 of the sequence below begins with a smaller, highest-leverage subset: Item 2 (Look-Ahead Bias), Item 3 (Data Integrity), Item 6 (Traceability), and Item 18 (Read-Only Market Access). A failure in any of those four would undermine trust in every other finding — the first three because they determine whether the engine's outputs mean anything, and Item 18 because if the running process holds a credential with trade permissions, then Item 1 is aspirational no matter what the code currently does. This reorders Step 3 internally — it does not change the sequence itself.

New — who performs the audit. The audit sequence has always specified what gets checked and how findings get recorded, but never who does the checking. It does now, because the answer was the weakest part of the whole arrangement: the party that co-drafted these rules and helped build the engine cannot also be the party that declares it compliant. Steps 3, 4, and 8 belong to an independent auditor[46] — a separate model, commissioned for the purpose, that wrote neither this document nor the code it is judging. Viktor has named the intended auditor: Grok, from xAI, once a version he judges capable of the task has released. That choice is stated here as the current plan, not as a fifth thing this document requires — the actual requirement is the one above, a model with

no stake in this engine and no shared authorship, and if Grok isn't ready or isn't equal to the task when the time comes, the requirement stands and the specific choice is free to change without touching a rule.

The roles are separated deliberately, and the asymmetry is the point. Claude prepares the audit package and writes none of the findings. The independent auditor evaluates the engine against the complete 44-rule register and produces the findings. Claude then answers each finding adversarially, including — especially — every finding against its own work. Viktor adjudicates, and where the auditor and Claude disagree, that disagreement stays visible to him rather than being resolved inside either party's head. Neither side gets to declare itself correct: the builder cannot self-certify, and the auditor does not receive unilateral authority either.

What independence here does and does not mean. The auditor is independent in authorship, not presumed independent in judgment[47]. It has no stake in this engine looking good, which is the specific problem being solved. It is not an oracle: it shares training data and reasoning habits with the assistant it is checking, so it can repeat a mistake with complete confidence. A clean audit is therefore evidence, not proof — the same correction this document already had to make about four reviewers agreeing with each other. Any reading of the audit that treats auditor agreement as validation has re-introduced the exact fallacy Revision 3 removed.

This isn't hypothetical. Reviewer 4 — credited earlier in “Improvements Made” as the sharpest and most adversarial of the four — turned out to be Claude, Opus 5, in a separate session with no memory of co-drafting this document. It had real authorship independence: it didn't write these rules or this engine, and it audited conduct the other three reviewers never thought to check. It did not have judgment independence in any strong sense — same base model family as the assistant whose conduct it was auditing, same as the one writing this sentence. That it still found a real defect is evidence the split is worth having. It is not evidence that a fresh Claude session is sufficient for Steps 3, 4, and 8. If the independent auditor commissioned for the actual audit turns out to be Claude as well, this paragraph is the reason that should be treated as a weaker result than a genuinely different model family, not a redundant precaution.

Viktor's answer to that risk is above: Grok, a genuinely different model family from Claude, built by a different company on different training data. That satisfies the exact concern this paragraph raises, and it's worth saying so plainly rather than leaving the concern hanging after the decision that resolves it. What it doesn't satisfy on its own is the earlier requirement — a model with no stake in the outcome and no shared authorship. Different lineage is necessary for judgment independence; it was never sufficient by itself for the whole thing Steps 3, 4, and 8 need.

What the audit package must contain. The auditor receives the artifact and the evidence, not the Constitution plus a description of what the engine supposedly does — that is what makes Revision 3's “demand the artifact, not the argument” safeguard operational rather than aspirational. The package includes the actual source of all sixteen modules, real inputs and outputs, and enough failure history to make known *classes* of failure visible: that a semantics mismatch once broke fourteen modules at once, that backtesting took down a previous build, that this is the third build. What it must not contain is the conclusions the auditor is supposed to reach independently. Telling it where to look is legitimate; telling it what it will find is contamination, and produces an auditor that confirms hints rather than one that tests them.

This creates one tension worth naming, because the document itself causes it. The two “Already known” items above — Prediction V1's epistemic status, and Layer 5 classified as Unknown for possible circular reasoning — are conclusions of exactly the kind the paragraph above warns against, and the auditor cannot avoid seeing them, since it has to read this section to audit against it. The resolution is to hand them over as open questions it is required to test independently rather than as findings to confirm, and to ask it explicitly what *other* unknowns exist that nobody here has flagged. If the audit comes back having confirmed exactly these two and found nothing else,

that result should be read as a warning about the audit, not as a clean bill of health for the engine.

New — how a finding gets recorded. Each audit finding, once made, should record the following fields, so findings are comparable to each other rather than free-text notes of varying detail:

Field	What goes here
Principle	Which Tier and item number the finding is checking against.
Status	Compliant / Non-compliant / Unknown[34] — never a guess dressed up as one of the first two.
Severity[35]	Critical — violates a Tier 1 invariant in a way that could reach a live decision or a customer. Major — violates a Tier 1 invariant but is contained or not yet reachable. Moderate — a Tier 2 or 3 gap that weakens the system without directly breaking an invariant. Minor — cosmetic, cleanup, or a Tier 4 preference. Kept independent of Status: an Unknown finding can still be rated Critical.
Evidence	The specific code, output, or test result the finding is based on — something Viktor (or a future second reviewer) can check directly without having to re-run or trust Claude's reasoning. A log line, a diff, a test result, a screenshot of the panel — not a paragraph of argument standing in for one.
Impact	What actually goes wrong if this finding is left unaddressed.
Required action	What needs to change, in concrete terms — or “none” if the finding is Compliant.
Effort	Small / Medium / Large — a rough sizing of the work the required action takes, independent of Severity. Used together with Severity to prioritize in Step 5 below.
Verification	How it will be confirmed the required action actually worked.
Re-audit	Whether and when this item gets checked again after the fix.

New — two working rules for the audit. First: “confidence is not probability[36]” unless it has been specifically, empirically calibrated as one — a lens for catching outputs that look statistically precise without having earned that precision. Second: Item 9's distinction between a raw measurement, a derived quantity, a heuristic score, a model output, and an interpretation gets resolved during the audit itself, under Item 8 (Epistemic Honesty) — a scope decision to make once real code is in front of it, not a document change to make now.

New — audit independence safeguards. Claude holds the technical and architectural judgment calls, wrote much of the engine, co-drafted these rules, and is also the party positioned to audit the engine against them. That's a real conflict this document didn't previously name, and naming it doesn't resolve it by itself — these three safeguards are the mitigation. First, every finding's Evidence field must be independently verifiable without re-running Claude's reasoning (see the Evidence row above). Second, the Minimum Viable Audit gate items (Items 2, 3, 6) get checked a second time by a reviewer who did not write the code under review — Viktor himself, another AI given the same code and no prior context, or a future collaborator — before being marked Compliant. Third, where Claude marks something Compliant, Viktor is entitled to demand the underlying artifact — the actual log, diff, or test output — not the written argument for why it's compliant.

New — what “the audit has run” means. The scope freeze[4] lifts on this specific, checkable condition: every Tier 1 invariant has a recorded finding — Compliant, Non-compliant, or Unknown — using the schema above, regardless of whether the fixes for any Non-compliant findings have actually landed yet. That's the point at which

the engine has told this document something it doesn't yet cover; fixing what the audit finds is Steps 6 through 8 below, and doesn't have to finish first.

New — what ratification means. Step 1 below gates everything after it, but had no stated act that actually performs it. It now does: ratification[3] happens the moment Viktor writes “ratified” and a dated row is added to Version History. Nothing heavier is required.

The full sequence, with the party responsible for each step named explicitly: 1) Principles v1.0 — *Viktor* reviews and ratifies this document, with any edits (see “What ratification means” above). 2) Version/freeze — it's version-controlled from that point forward; any future revision is a new, dated version, not a silent edit, and the principles themselves stop expanding while the audit runs. 2a) Audit package — *Claude* assembles the code, evidence, and failure history described above, and writes no findings. 3) Audit the current engine — the *independent auditor* works through the Minimum Viable Audit gate above, then module by module against all four tiers. 4) Record a real finding for each item checked — *independent auditor*, using the structure above. 4a) Adversarial response — *Claude* answers every finding, including each one against its own work; unresolved disagreements go to *Viktor* to adjudicate. 5) Prioritize findings by severity and effort. 6) Fix deliberately, one controlled change at a time, per Tier 3. 7) Regression test each fix. 8) Re-audit the items that changed — *independent auditor* again, not a self-check by whoever made the fix. 9) Only then — with the safety net actually in place, and the freeze lifted per “what the audit has run means” above — build the backtesting architecture, carefully.

Future Amendment Candidates (Not Yet Adopted)

Copilot drafted six complete candidate principles for a future v1.1, on its own initiative. None of these are part of this document's frozen scope[4] — nothing below has been adopted, and nothing below requires action now. They're recorded here, per Tier 3's own documentation-of-decisions principle, so they aren't lost before the audit has run far enough to make revisiting them worthwhile.

Candidate	Proposed tier	Gist
Explicit Observability & Logging	Tier 2	Every module's decision-relevant state should be inspectable at runtime, not just reconstructable after the fact.
Architectural Decision Records (ADR)[29]	Tier 3	Significant design decisions get a short, dated written record of what was decided and why — a concrete mechanism for Tier 3's existing documentation principle.
Predictive Component Validation Protocol	Tier 3	A defined, repeatable procedure for moving any feature from “computationally validated” to “empirically validated” status — the missing operational half of Tier 1, Item 7.
Failure Classification & Reporting	Tier 3	A standard way of categorizing and recording failures when they happen, closely related to the audit-finding schema now in the Next Steps section.
Backtesting Environment Isolation Contract[25]	Tier 2	A concrete technical contract — not just a stated goal — for how backtesting stays sandboxed from the live engine. The implementation-level companion to Tier 1, Item 17.

Candidate	Proposed tier	Gist
Predictability Over Raw Performance	Tier 4 (optional)	A stability-flavored preference for the engine behaving predictably over squeezing out marginal performance — offered as optional since it may already be covered by existing Tier 4 preferences.
A Written Kill Condition[42]	Tier 3	A condition, stated before a backtest runs, under which a specific edge or the engine itself gets abandoned — the falsifiability half of Tier 1, Item 7 that nothing currently requires in writing.
Confidence Calibration Log[43]	Tier 2	Every confidence value the engine emits, logged against what actually happened, plotted as a reliability curve — the concrete mechanism that would make "confidence is not probability" a checked fact instead of a definition.
Edge-Persistence Hypothesis Requirement	Tier 1 (companion to Item 7)	Every claimed edge documents who is reliably on the other side of the trade and why they keep taking it — an edge with no survival story is usually a fit to noise that hasn't been embarrassed yet.
UI-Level Enforcement of Item 1	Tier 2	The panel shows what was observed, what was inferred, what remains unknown, and what would change the read — not a verdict and a number. Item 1 can be satisfied in code and violated in the interface; this closes that gap.
Two-Analyst Lens-vs-Property Test	Tier 3 (audit methodology)	For any Item 9 quantity: would two competent analysts with different reasonable parameter choices get materially different answers? If yes, it's a lens, and the panel shouldn't render it with the same visual weight as a price. Feeds directly into the term registry work already logged in Engineering Notes, Entry #6.

Two of these — Backtesting Environment Isolation Contract and Failure Classification & Reporting — are implementation-level companions to things already in this document (Item 17 and the audit-finding schema, respectively) rather than new territory. That overlap is itself a small piece of independent validation that those two existing items are pointing at something real. The final five candidates — Kill Condition through the Lens-vs-Property Test — came from a philosophical input Viktor brought in Revision 4, not from Copilot; see Phase7_Tier0_Companion.pdf for the reasoning behind each.

Version History

Version	Date	Status	Notes
v1.0	August 25, 2026	Draft — superseded by Rev. 1 below	Initial synthesis from the principles discussion between Viktor and Claude. Scope frozen at 17 Tier 1 invariants, 7 Tier 2 principles, 10 Tier 3 process items, and 6 Tier 4 preferences.
v1.0, Rev. 1	August 25, 2026	Draft — superseded by Rev. 2 below	Revised after independent review by Copilot, Gemini, and ChatGPT: Tier 0 wording fixed, audit sequence given a starting gate and a finding schema, two audit scoping notes added, an amendment-process note added, six future candidates logged in a new appendix, and an explanatory glossary added throughout. Scope unchanged — still 17 / 7 / 10 / 6.
v1.0, Rev. 2	August 25, 2026	Draft — superseded by Rev. 3 below	Added “The Scope Freeze” — a dedicated explanation, in How This Document Works, of what the frozen-scope rule actually means, why it exists (tied to epistemic honesty, Item 8), what it does and doesn't cover, and when it lifts. Explanatory addition only, requested directly by Viktor — scope unchanged, still 17 / 7 / 10 / 6.
v1.0, Rev. 3	August 26, 2026	Draft — superseded by Rev. 4 below	Revised after a fourth, independent review. Fixed a self-referential evidence claim on the Improvements page (reviewer agreement was presented as validation; corrected to state it as a plausibility check). Item 5 changed from “should” to “must,” the only Tier 1 item not phrased as an invariant. Items 4 and 12's escape hatches given audit-time guidance (exceptions only count if documented before the audit began). Defined the freeze-lifting trigger, the act of ratification, severity-level rubrics, and added an Effort field to the finding schema. Added three audit-independence safeguards addressing the fact that Claude co-drafted these rules and also audits against them. Scope unchanged, still 17 / 7 / 10 / 6.

Version	Date	Status	Notes
v1.0, Rev. 4	August 26, 2026	Draft — superseded by Rev. 5 below	Revised after a philosophical input on epistemics and market structure. Sharpened the rationale text under Tier 3's Controlled Changes (the Duhem-Quine problem) and Tier 1, Item 14 (ergodicity, cross-referenced to the risk-tolerance feature idea in Engineering Notes, Entry #5). Added five new Future Amendment Candidates: a written kill condition, a confidence calibration log, an edge-persistence hypothesis requirement, UI-level enforcement of Item 1, and a two-analyst lens-vs-property audit test. A new companion document, Phase7_Tier0_Companion.pdf, holds the philosophical material that sits above the engine rather than inside it. Scope unchanged, still 17 / 7 / 10 / 6. Not yet ratified.
v1.0, Rev. 5	August 26, 2026	Draft — superseded by Rev. 6 below	Corrects a scope drift Viktor caught in Revision 4's Item 14 rationale, which had drifted toward implying the engine should size positions or recommend money amounts. Item 14's rationale, and the Revision 4 summary describing it, are both corrected: ergodicity is the reason entry/stop/target price geometry deserves as much care as the directional read, not a reason for the engine to touch position sizing — that stays the trader's decision alone, per Item 1, regardless of the risk-tolerance percentage the trader configures. No wording elsewhere required correction. Scope unchanged, still 17 / 7 / 10 / 6. Not yet ratified.

Version	Date	Status	Notes
v1.0, Rev. 6	August 26, 2026	Draft — superseded by Rev. 7 below	SCOPE CHANGED — the first revision since v1.0 to do so. Added four Tier 1 credential-security invariants at Viktor's direction: Item 18 (Read-Only Market Access, stated categorically rather than as a default), Item 19 (Withdrawal Permissions Never Enabled, kept separate as a floor surviving any future amendment to Item 18), Item 20 (Credentials Never Exposed), and Item 21 (Operator Credentials Stay With the Operator). Item 18 converts Item 1 from a behavioral promise into a structural property by moving the guarantee into the exchange's permission model, and joins Items 2, 3, and 6 in the Minimum Viable Audit gate. Also named an independent auditor as the party responsible for Steps 3, 4, and 8, added Step 2a (Claude prepares the package, writes no findings) and Step 4a (Claude answers every finding adversarially, Viktor adjudicates), distinguished authorship independence from judgment independence, and specified that the audit package carries failure classes rather than conclusions. Operational credential detail moved to a new companion, Phase7_Credential_Security_Protocol.pdf. This scope change is legitimate only because the freeze begins at ratification, which has not occurred; after ratification a change of this size requires the audit to run and find a gap first. Scope now 21 / 7 / 10 / 6 — a 44-rule register. Closes Engineering Notes Entries #1 and #6. Not yet ratified.
v1.0, Rev. 7	August 26, 2026	Draft — superseded by Rev. 8 below	No rule changed. Viktor confirmed Reviewer 4's identity — Claude, Opus 5, in a separate chat session with no memory of co-drafting this document — and Revision 6's account of Reviewer 4 in “Improvements Made” is updated to credit it by name. The independent-auditor discussion in Next Steps is sharpened with the direct consequence: Reviewer 4 is the documented case of real authorship independence without strong judgment independence, and if the Step 3/4/8 auditor also turns out to be Claude, that result should be weighted as weaker than a genuinely different model family would produce. Register unchanged, still 21 / 7 / 10 / 6. Not yet ratified.

Version	Date	Status	Notes
v1.0, Rev. 8	August 26, 2026	Draft — pending review and ratification	No rule changed. Viktor decided the independent auditor for Steps 3, 4, and 8 will be Grok, from xAI, once a version he judges capable has released — recorded in Next Steps as the current operational plan, not folded into the frozen requirement, which stays general (no stake, no shared authorship) so a future substitution needs no amendment. Grok answers Revision 7's warning on the lineage half — genuinely different company and training data from Claude — but the no-stake requirement still has to hold regardless of which model is chosen. Step 3 cannot begin until Grok releases; Step 2a, Claude assembling the audit package, is not blocked by that and can proceed now. Register unchanged, still 21 / 7 / 10 / 6. Not yet ratified.
RATIFIED	August 26, 2026	Ratified — v1.0 now in effect	No rule changed. Viktor reviewed Revision 8 — the document itself, plus a curated 12-page reading excerpt and a plain-language Swedish companion covering the three judgment calls made on his behalf (Items 18, 19, 21) — and wrote: “I have read the document and i APPROVE!” Per “What ratification means” in Next Steps, that statement is recorded here as the ratifying act, since no heavier process than a dated Version History row is required. The scope freeze is now in force for the reason it was always going to take effect: no invariant, principle, process item, or preference changes again until the audit runs and finds a gap. Register frozen at 21 / 7 / 10 / 6. Step 1 of the audit sequence is complete; Step 2a (Claude assembling the audit package) may proceed now. Step 3 remains blocked on Grok's release, per the Rev. 8 row above.
LICENSED	August 26, 2026	Ratified — v1.0, licence added for publication	No rule changed, and no rule text touched. Viktor decided to publish this document publicly, and a licence line was added to the title page so the terms travel with the file rather than living only in the repository that hosts it — a PDF is separated from its README the first time someone forwards it. Terms: CC BY 4.0, which permits reuse and adaptation including commercially, with attribution as the only condition. The engine's source code is licensed separately under MIT; Creative Commons licences are not intended for software and the split is deliberate. Recorded as its own dated row rather than folded into the RATIFIED row above, because that row describes an act that is finished and this document does not edit its own history. Register unchanged and still frozen at 21 / 7 / 10 / 6.